

```
import pandas as pd

# Download Titanic Dataset
url = "https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv"
df = pd.read_csv(url)

df.head()
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	7140764293
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Thayer)	female	38.0	1	0	PC 17596
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STC 3101282
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	315164

Next steps:

[Generate code with df](#)

[New interactive sheet](#)

```
# Select important features

features = ['Pclass', 'Sex', 'Age', 'Fare']
target = 'Survived'

# Keep only required columns
data = df[features + [target]]

# Fill missing Age values with average age
data['Age'] = data['Age'].fillna(data['Age'].mean())

# Convert Male/Female to numbers
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})

# Display cleaned data
data.head()
```

```
/tmp/ipykernel_5423/2434712978.py:10: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: <https://pandas.pydata.org/pandas->

```
data['Age'] = data['Age'].fillna(data['Age'].mean())
```

```
/tmp/ipykernel_5423/2434712978.py:13: SettingWithCopyWarning:  
A value is trying to be set on a copy of a slice from a DataFrame.  
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: <https://pandas.pydata.org/pandas->

```
data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
```

	Pclass	Sex	Age	Fare	Survived
0	3	0	22.0	7.2500	0
1	1	1	38.0	71.2833	1
2	3	1	26.0	7.9250	1
3	1	1	35.0	53.1000	1
4	3	0	35.0	8.0500	0

Next steps:

[Generate code with data](#)[New interactive sheet](#)

```
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
  
# Features (inputs)  
X = data[['Pclass', 'Sex', 'Age', 'Fare']]  
  
# Target (output)  
y = data['Survived']  
  
# Split dataset into training and testing data  
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=42  
)  
  
# Create and train model  
model = LogisticRegression(max_iter=1000)  
model.fit(X_train, y_train)  
  
print("Model Training Completed Successfully!")
```

Model Training Completed Successfully!

```
from sklearn.metrics import accuracy_score, classification_report  
  
# Make predictions  
y_pred = model.predict(X_test)  
  
# Calculate accuracy  
accuracy = accuracy_score(y_test, y_pred)  
  
print("Model Accuracy:", accuracy)
```

```
# Detailed Report
print("\nClassification Report:")
print(classification_report(y_test, y_pred))
```

Model Accuracy: 0.7988826815642458

```
Classification Report:
              precision    recall  f1-score   support

     0       0.82         0.85         0.83         105
     1       0.77         0.73         0.75          74

 accuracy          0.80         0.80         0.80         179
 macro avg         0.79         0.79         0.79         179
 weighted avg         0.80         0.80         0.80         179
```